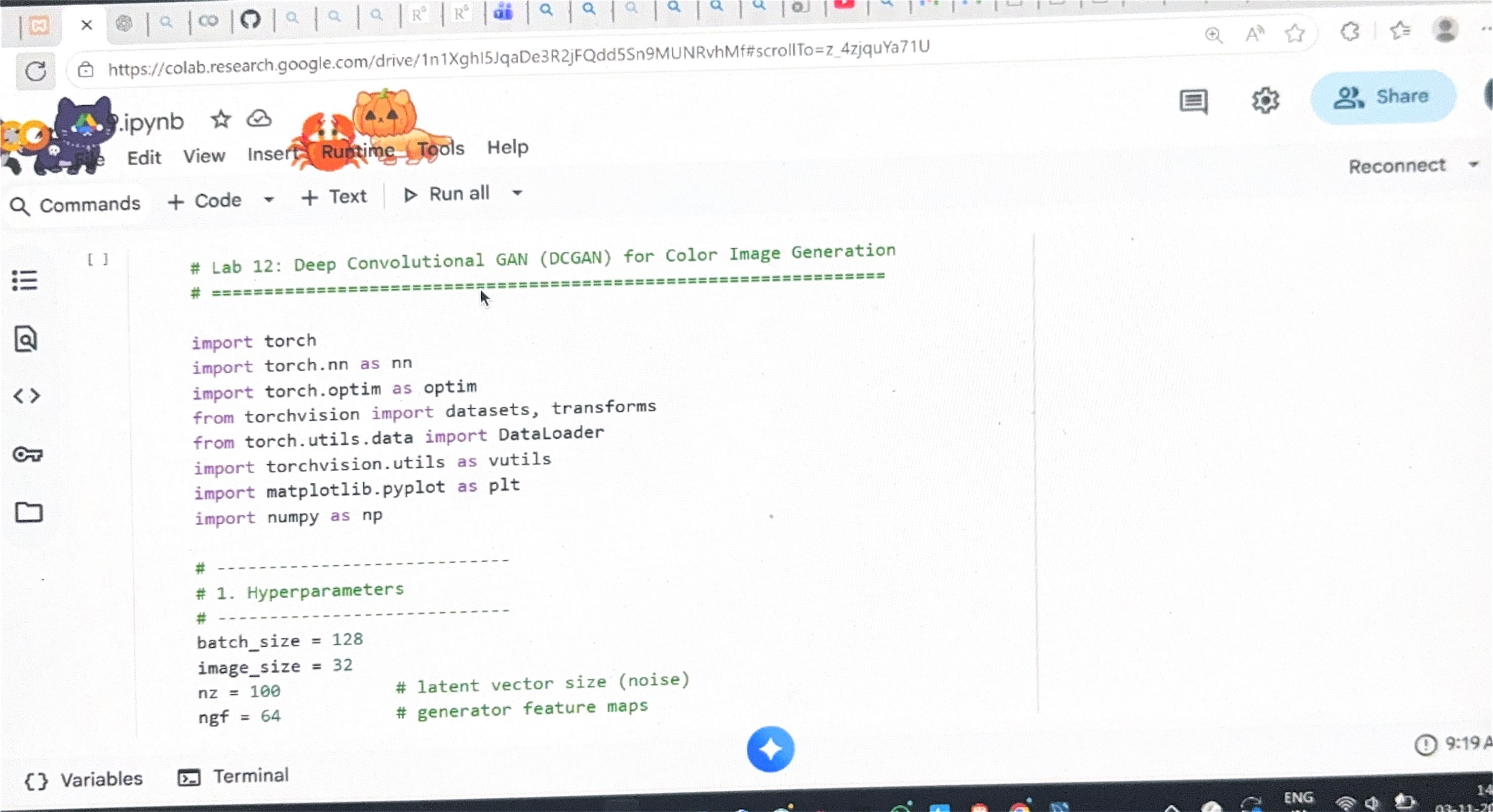




https://colab.research.google.com/drive/1n1XghI5JqaDe3R2jFQdd5Sn9MUNRvhMf#scrollTo=z_4zjqyYa71U

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Share

Reconnect

```
[ ]  
# Lab 12: Deep Convolutional GAN (DCGAN) for Color Image Generation  
# =====
```

```
import torch  
import torch.nn as nn  
import torch.optim as optim  
from torchvision import datasets, transforms  
from torch.utils.data import DataLoader  
import torchvision.utils as vutils  
import matplotlib.pyplot as plt  
import numpy as np
```

```
# -----  
# 1. Hyperparameters  
# -----  
batch_size = 128  
image_size = 32  
nz = 100          # latent vector size (noise)  
ngf = 64          # generator feature maps
```

Variables Terminal

9:19 A



9.ipynb

File Edit View Insert Runtime Tools Help

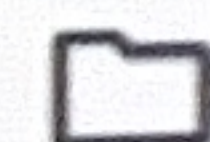
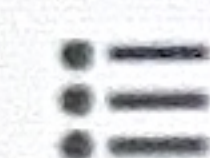


Share

N

Commands + Code + Text Run all

Reconnect

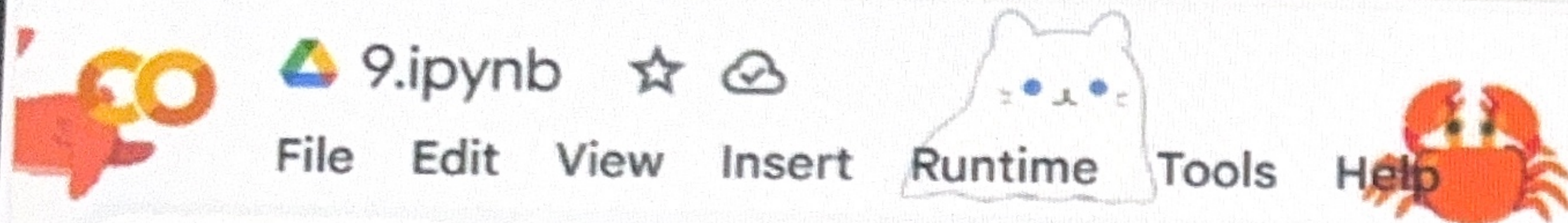


```
ndf = 64          # discriminator feature maps
nc = 3            # RGB
lr = 0.0002
beta1 = 0.5
epochs = 5
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# -----
# 2. Dataset
# -----
transform = transforms.Compose([
    transforms.Resize(image_size),
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

dataset = datasets.CIFAR10(root="./data", download=True, transform=transform)
dataloader = DataLoader(dataset, batch_size=batch_size, shuffle=True)
```

```
# -----
```

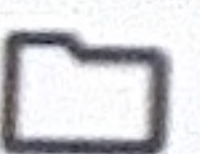
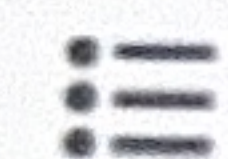
File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all



Share

Reconnect



```
# -----  
# 3. Generator  
# -----  
class Generator(nn.Module):  
    def __init__(self):  
        super(Generator, self).__init__()  
        self.main = nn.Sequential(  
            nn.ConvTranspose2d(nz, ngf*8, 4, 1, 0, bias=False),  
            nn.BatchNorm2d(ngf*8),  
            nn.ReLU(True),  
            nn.ConvTranspose2d(ngf*8, ngf*4, 4, 2, 1, bias=False),  
            nn.BatchNorm2d(ngf*4),  
            nn.ReLU(True),  
            nn.ConvTranspose2d(ngf*4, ngf*2, 4, 2, 1, bias=False),  
            nn.BatchNorm2d(ngf*2),  
            nn.ReLU(True),  
            nn.ConvTranspose2d(ngf*2, ngf, 4, 2, 1, bias=False),  
            nn.BatchNorm2d(ngf),  
            nn.ReLU(True),  
            nn.ConvTranspose2d(ngf, nc, 3, 1, 1, bias=False),
```





9.ipynb ☆ ☁

File Edit View Insert Runtime Tools Help

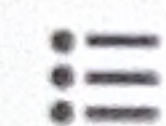


Share

N

🔍 Commands + Code + Text ▶ Run all

Reconnect ^



```
def forward(self, input):
    return self.main(input)

# -----
# 4. Discriminator
# -----
class Discriminator(nn.Module):
    def __init__(self):
        super(Discriminator, self).__init__()
        self.main = nn.Sequential(
            nn.Conv2d(nc, ndf, 4, 2, 1, bias=False),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf, ndf*2, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*2),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf*2, ndf*4, 4, 2, 1, bias=False),
            nn.BatchNorm2d(ndf*4),
            nn.LeakyReLU(0.2, inplace=True),
            nn.Conv2d(ndf*4, 1, 4, 1, 0, bias=False),
```




```
nn.Sigmoid()
)

def forward(self, input):
    return self.main(input).view(-1)

# -----
# 5. Initialize models and optimizers
# -----
netG = Generator().to(device)
netD = Discriminator().to(device)

criterion = nn.BCELoss()
optimizerD = optim.Adam(netD.parameters(), lr=lr, betas=(beta1, 0.999))
optimizerG = optim.Adam(netG.parameters(), lr=lr, betas=(beta1, 0.999))

fixed_noise = torch.randn(64, nz, 1, 1, device=device)

# -----
# 6. Training with Loss Tracking
```



9:19 AM


```
# -----  
# 7. Plot Loss Graph  
# -----
```


9.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

Reconnect

```
plt.figure(figsize=(8,5))
plt.plot(G_losses, label="Generator Loss (G)", marker='o')
plt.plot(D_losses, label="Discriminator Loss (D)", marker='s')
plt.title("Generator vs Discriminator Loss During Training")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.legend()
plt.grid(True)
plt.show()

# -----
# 8. Show Final Generated Images
# -----
with torch.no_grad():
    fake = netG(fixed_noise).detach().cpu()
    grid = vutils.make_grid(fake, padding=2, normalize=True)
    plt.figure(figsize=(8,8))
    plt.axis("off")
    plt.title("Generated Images (After Final Epoch)")
```

Variables

Terminal

9:19 AM



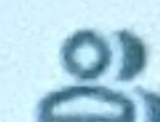
File Edit View Insert Runtime Tools Help

 ShareReconnect

```

Epoch [1/5] Loss_D: 0.5758 Loss_G: 3.0092
Epoch [2/5] Loss_D: 1.0516 Loss_G: 1.7795
Epoch [3/5] Loss_D: 0.9452 Loss_G: 2.0873
Epoch [4/5] Loss_D: 1.2850 Loss_G: 3.5633

```

Share

N

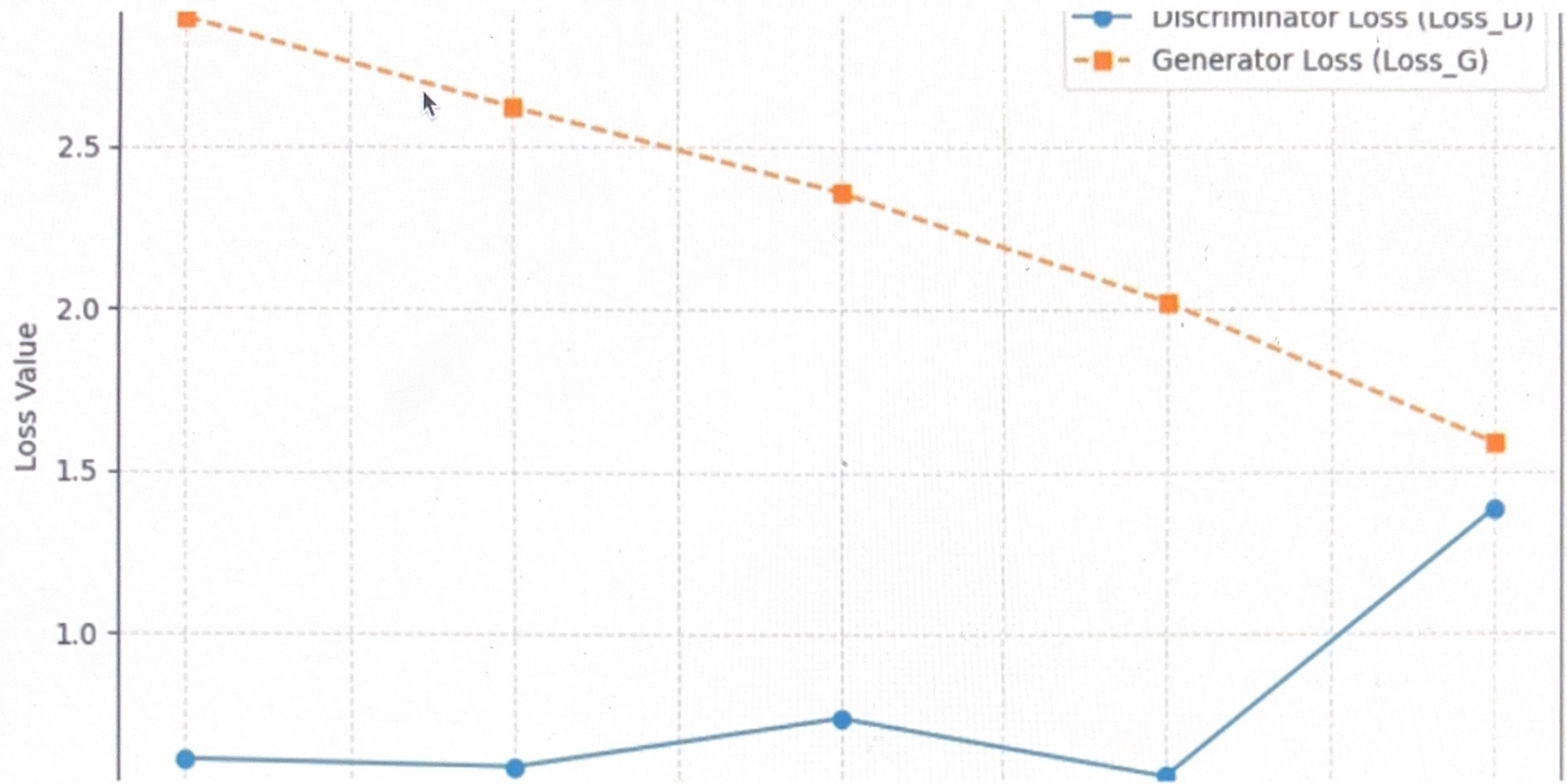
Reconnect

```
epochs = [1, 2, 3, 4, 5]
loss_d = [0.6131, 0.5894, 0.7406, 0.5590, 1.3862]
loss_g = [2.9012, 2.6182, 2.3580, 2.0222, 1.5855]

# Plot losses
plt.figure(figsize=(8,5))
plt.plot(epochs, loss_d, marker='o', linestyle='-', label='Discriminator Loss (Loss_D)')
plt.plot(epochs, loss_g, marker='s', linestyle='--', label='Generator Loss (Loss_G)')

# Labels and title
plt.title("GAN Training Progress")
plt.xlabel("Epoch")
plt.ylabel("Loss Value")
plt.legend()
plt.grid(True, linestyle='--', alpha=0.6)
plt.tight_layout()
plt.show()
```





12. Implement a Deep convolutional GAN to generate complex

Aim:-

- color images

To implemented deep convolutional Generative Adversarial Network (DCGAN) that generates complex color images from random noise.

Objective:-

- To understand the working of GAN's
- To implement convolutional and transposed convolutional layer for image generation.
- To train generator & discriminator networks to produce realistic color images.
- To visualize & evaluate the quality of generated images during & after training.

Pseudocode:-

1. Import necessary libraries.
2. Load & preprocess dataset (CIFAR-10)
 - Normalize images pixel to range $(-1, 1)$
3. Define Generator network
 - input: random noise vector (2)
 - layer: Dense $\rightarrow$ Reshape $\rightarrow$ convtranspo 20 $\rightarrow$ batch Norm $\rightarrow$
 - output: 3 channel color image with tanh activation.
4. Define discriminator network:
 - input: image (real / false)
 - layer: conv20 $\rightarrow$ leakyRelu $\rightarrow$ dropout
 - output: probability (real / false) using sigmoid activation.

5. Define loss function.

- Generator loss : encourage $\mathbb{E}[D(G(z))] \rightarrow 1$
- Discriminator loss : distinguish real vs fake correctly.

6. Initialize optimizer.

7. for each training ~~step~~ epoch:

- Train Discriminator on real image (label = 1) fake image (label = 0)
- Train Generator to fool the discriminator (label = 1)
- save sample generator image every few epochs.

8. Displays loss curves & generated image samples.

9. End training when generated images become visually realistic.

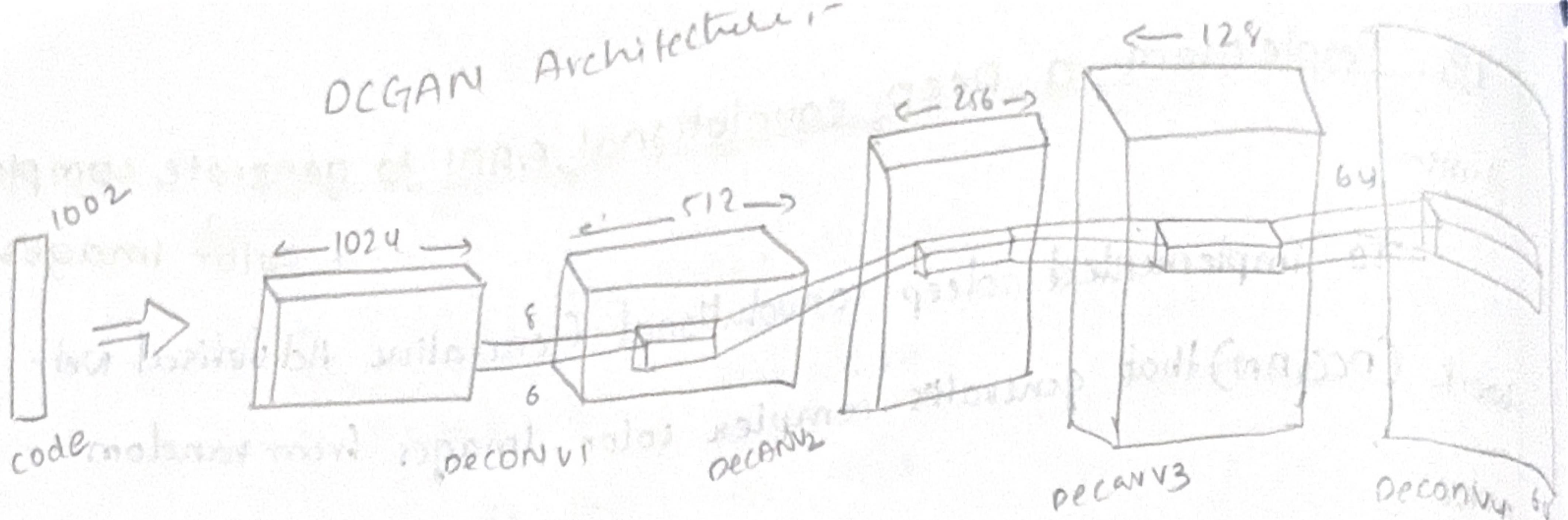
Observation:-

- This discriminator initially learns faster & easily detects fake image.
- The generator improves gradually, producing sharper & more colorful images over epochs.
- After several epochs, both networks reach a balanced state & generated images closely resemble real samples.

Result:-

~~Not~~ Successfully implemented a deep convolutional GAN to generate complex color images.

DCGAN Architecture



Epoch (1/5) loss D : 0.6131 loss G : 2.9612
 Epoch (2/5) loss D : 0.5894 loss G : 2.6112
 Epoch (3/5) loss D : 0.4006 loss G : 2.3580
 Epoch (4/5) loss D : 0.6890 loss G : 2.0222
 Epoch (5/5) loss D : 1.3862 loss G : 1.5855

